

The ROS 2 Computation Graph

Nodes, messages, discovery, and the setting that decides whether anything arrives

Prof. Anis Koubaa

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 2, Lecture A



What this lecture does

Roadmap

Last week we argued that robot software must be many small programs exchanging messages. Today: what those programs are, what the messages are, how they find each other, and why a correct-looking pair sometimes exchanges nothing at all.

- 1 Nodes and the graph
- 2 Messages: the contract
- 3 How nodes find each other
- 4 Quality of Service
- 5 Workspaces, packages, and building

Nodes and the graph

Where we left off

The naive robot program failed for five structural reasons:

The problem	The answer we are about to build
One rate for everything	Each node has its own timer
One crash kills everything	Nodes are separate OS processes
One machine	Nodes discover each other over the network
Nothing is reusable	Nodes agree on message <i>types</i> , not on each other
You cannot see inside it	Every message is inspectable from outside

Today is the vocabulary for all five. **Session B is where you type it.**

The node: the unit of a robot program

A **node** is one process that does one job.

Good nodes

- `lidar_driver` — talk to the LiDAR, publish scans
- `controller` — turn a path into wheel velocities
- `battery_monitor` — watch the voltage, warn

A node that will hurt you

- `robot` — reads sensors, plans, controls, logs, and draws the user interface

It works. Then the planner gets slow, and your control loop goes with it.

The test

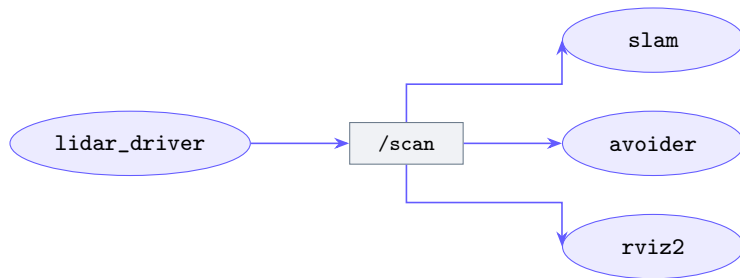
If you cannot say what a node does in one short sentence without using “and”, it is two nodes.

Four ways nodes talk

Mechanism	Shape	Use it for	Example
Topic	one-way stream, many-to-many, no-body waits	continuous data	/scan, /odom, /cmd_vel
Service	request → reply, caller waits	a quick question	“reset the odometry”
Action	goal, feedback, result — cancellable	long jobs	“navigate to the door”
Parameter	a named value the node reads, and you can change	configuration	max_speed = 0.22

Topics carry almost all robot traffic — and all of Weeks 4–6. Actions arrive properly in Week 11, when you ask Nav2 to drive somewhere and it takes forty seconds.

Topics: nobody knows who is listening

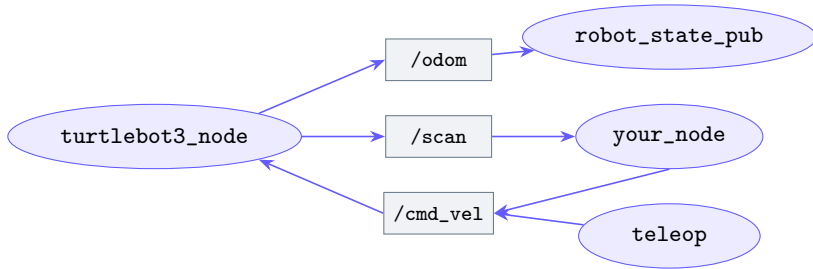


`lidar_driver` publishes and moves on. It does not know three nodes are listening, and behaves identically if none are.

Why this matters more than it looks

Adding a fourth listener needs **no change to the publisher** — which is how you attach a recorder or a visualiser to a running robot without restarting the code that is driving it.

A real graph: TurtleBot3, doing nothing yet



Six processes, started separately, that have never been compiled together. Two of them (`your_node` and `teleop`) publish to the same topic — and the hardware node accepts both, because a topic is many-to-many.

Which is also a bug waiting to happen. Two things commanding one robot is exactly as bad as it sounds. Week 11 is partly about arbitrating that.

Messages: the contract

A message is a typed contract

A topic is not a pipe for arbitrary bytes. It carries **one declared type**, and both sides are checked.

`geometry_msgs/msg/Twist`

```
Vector3  linear      # x, y, z   metres / second
Vector3  angular     # x, y, z   radians / second
```

For a differential-drive robot only two of those six numbers mean anything: `linear.x` (forward) and `angular.z` (turn). The rest must be zero — a wheeled robot cannot move sideways.

Under the hood

The light box means **not runnable**. This is an interface definition, compiled into Python and C++ classes at build time. That is why a typo in a field name fails when you build, not at 3 a.m. on the robot.

The types you will meet this semester

Type	Topic	Carries
geometry_msgs/Twist	/cmd_vel	Commanded v and ω
sensor_msgs/LaserScan	/scan	One LiDAR sweep: ranges, angles, timestamps
nav_msgs/Odometry	/odom	Estimated pose and velocity, with covariance
sensor_msgs/Imu	/imu	Angular rate, linear acceleration
nav_msgs/OccupancyGrid	/map	The map, as a grid of cells

In the field

These are not this course's conventions — they are the field's. A robot from a different vendor publishes `sensor_msgs/LaserScan` on `/scan`. Learn the type once, and your obstacle-avoidance node works on hardware you have never seen.

Every message is stamped, and that is not a detail

Most sensor messages carry a `std_msgs/Header`: a timestamp and the name of the coordinate frame the data is expressed in.

<code>stamp</code>	when the measurement was <i>taken</i> — not when it arrived
<code>frame_id</code>	which frame it is measured in — <code>base_link</code> , <code>laser</code> , <code>map</code>

Truth 2, in a struct

Last week: nothing is instantaneous. The stamp is how a robot copes — it lets a node fuse a scan taken 100 ms ago with odometry taken 20 ms ago, *correctly*. Week 7 (TF2) and Week 9 (estimation) are built entirely on these two fields.

How nodes find each other

There is no central process

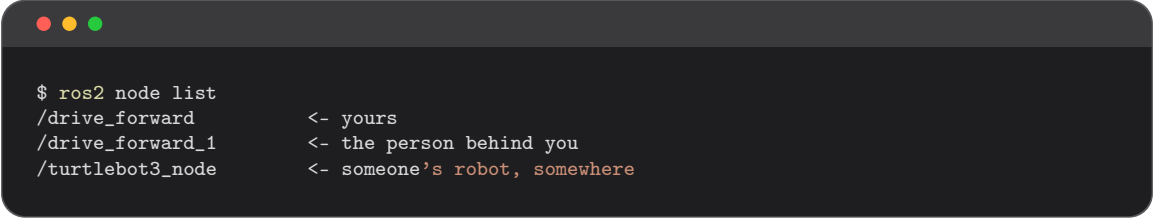
When a node starts, it announces itself on the network and listens for others doing the same. Publishers and subscribers on the same topic then connect **directly**.

	ROS 1	ROS 2
Finding each other	A central roscore kept the directory	Distributed discovery — no directory
If that dies	Nothing new can connect. The robot goes deaf.	There is nothing to die
Start-up order	Master first, always	Any order
Delivery guarantees	One behaviour, take it or leave it	Chosen per topic (next section)

This is the main reason ROS 2 exists, and the main reason it appears in products where ROS 1 largely did not. It is a rewrite, not an upgrade.

The consequence you met in Week 1

Discovery is **automatic** and reaches everyone on the network segment. In a shared lab, that is a problem:



```
$ ros2 node list
/drive_forward          <- yours
/drive_forward_1        <- the person behind you
/turtlebot3_node        <- someone's robot, somewhere
```

ROS_DOMAIN_ID partitions discovery. Same ID $\Rightarrow$ you see each other.

First diagnostic question of the entire semester: a topic is obviously being published and your node sees nothing — are you on the same domain ID?

Quality of Service

The setting everyone skips, and then loses a day to

A publisher and a subscriber must agree on **how** messages are delivered, not just on the type. That agreement is the QoS profile.

Setting	Options	What it decides
Reliability	RELIABLE / BEST_EFFORT	Retry lost messages, or let them go
Durability	VOLATILE / TRANSIENT_LOCAL	Does a late subscriber get the last message, or nothing until the next one?
History + depth	KEEP_LAST(n) / KEEP_ALL	How many to hold while the subscriber is busy

These are not performance knobs. They change whether your data arrives at all.

Choosing: it depends what the data is for

LiDAR scans — BEST_EFFORT

10 per second. If one is lost, another arrives in 100 ms.

Retrying a stale scan is worse than useless: it delays the fresh one behind it.

A map — TRANSIENT_LOCAL

Published once, rarely.

A node that starts late must still receive it — otherwise it waits forever for a message that already happened.

The rule of thumb

Streams of sensor data: best-effort, shallow queue. **State** that is published once and must not be missed: reliable and transient-local. Commands (/cmd_vel): reliable — a dropped stop command is a crash.

The failure that looks like nothing at all

Publisher is BEST_EFFORT. Subscriber demands RELIABLE. Both start cleanly, report no errors, and **nothing is ever delivered**.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of the command 'ros2 topic info /scan --verbose'.

```
$ ros2 topic info /scan --verbose
Publisher count: 1
  Reliability: BEST_EFFORT
Subscription count: 1
  Reliability: RELIABLE      <- incompatible: no connection
```

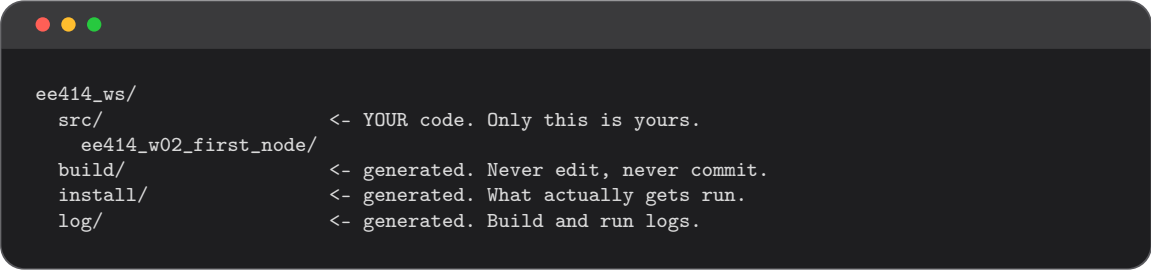
The subscriber asked for a guarantee the publisher does not offer, so the two are never matched. Not a bug — the system refusing to pretend.

In the field

Memorise `ros2 topic info --verbose`. When a subscriber is silent and the publisher is clearly publishing, nothing else answers the question this fast.

Workspaces, packages, and building

Where your code lives



```
ee414_ws/  
  src/                <- YOUR code. Only this is yours.  
    ee414_w02_first_node/  
  build/              <- generated. Never edit, never commit.  
  install/            <- generated. What actually gets run.  
  log/                <- generated. Build and run logs.
```

A **workspace** is a folder of packages you build together. A **package** is the unit you build, share and install — one folder, its own dependencies, its own nodes.

You edit `src/`. `colcon` produces the other three. If you are ever tempted to fix something in `build/`, the answer is in `src/`.

Build, then source. Always both.



```
$ cd ~/ee414_ws  
$ colcon build --symlink-install  
$ source install/setup.bash  
$ ros2 run ee414_w02_first_node talker
```

Sourcing is not optional and it does not persist. It edits the environment of *one shell*. A new terminal is a new shell, and knows nothing.

What sourcing actually does

It adds your `install/` to the paths ROS 2 searches. That is the entire mechanism behind “Package not found” when the package plainly exists — the shell was never told where to look.

Underlay and overlay

overlay — ~/ee414_ws/install : your packages



sourced first

underlay — /opt/ros/... : rclpy, Nav2, TF2, the standard messages

Source the underlay, then your workspace. Your overlay wins where names collide — which is how you can develop a modified version of a system package without uninstalling anything.

In the field

Sourcing the overlay in the wrong order, or sourcing two different workspaces into one shell, produces the strangest bugs in robotics: the code you are editing is not the code that runs. When something is inexplicable, open a clean terminal.

What session B does with all of this

- 1 Create a workspace and a package.
- 2 Write a publisher — a node that sends a message ten times a second.
- 3 Watch it from outside with `ros2 topic list`, `echo`, `hz`.
- 4 Write a subscriber, and see the two connect in `rqt_graph`.
- 5 Start both with a single launch file.
- 6 Break it on purpose, five ways, and fix each.

Bring a laptop with the setup check passing. Without it, you will watch.

Quiz 1 today — nodes, topics, the four mechanisms, and one QoS question.